Web-приложения и их архитектура. Objects. Context

Object & OOP basics in JS

Heap & Stack

Давайте кратко вернемся к хранению переменных по значению и по ссылке.

Важно понимать, что это не просто особенность языка (например, различия между объявлением функции и функциональным выражением), а серьезная техническая проблема, обусловленная низкоуровневой структурой компьютерных технологий.

Создание объекта

Итак, как вы помните, создание объекта в JS банальное:

```
const obj = { name: “Alex”, age: 18};
```

где name и age - поля объекта obj. Получение значений полей объекта происходит двумя способами:

```
1.   console.log(obj.name) // Expected output: Alex
2.   console.log(obj[‘name’]) // Expected output: Alex
```

Class и ключевое слово new

Классы в JS тоже создаются банально:

```
class Person {
      constructor (name, age) {
            this.name = name;
            this.age = age;
      }
}
```

Как можно заметить, в JS не обязательно напрямую прописывать поля класса вне конструктора.
Для создания объекта класса используется ключевое слово new:

```
const employee = new Person(“Alex”, 18);
```

Class и ключевое слово new

Новое ключевое слово выполняет ряд внутренних операций:

1) Неявное создание экземпляра

2) Запись свойств (выполнение самого конструктора)

3) Настройка прототипа

4) Неявный возврат экземпляра.

Создание объектов с использованием встроенных методов

1) `Object.create(prototype, ?properties)` создает новый объект, устанавливает указанный прототип и записывает указанные свойства. Список свойств указывается в формате имя: дескрипторы.

2) `Object.assign(target, ...sources)` создает новый объект, который является результатом копирования всех свойств источника в целевой объект.

```
const person : {}  = Object.create( o: {},  properties: {
    name: {
        enumerable: true,
        configurable: false,
        value: 'Max'
    }
});

const roman : {...} & {...}  = Object.assign(
    target: {},
    source1: { name: 'Roman' },
    source2: { age: 25 }
)
```

Остальные встроенные методы

1) `Object.keys(object)` — возвращает массив ключей перечисляемых свойств самого объекта.

2) `Object.values(object)` — возвращает массив значений перечисляемых свойств самого объекта.

3) `Object.entries(object)` — возвращает массив перечисляемых свойств самого объекта в виде пар [ключ, значение].

4) `Object.is(value1, value2)` — определяет, одинаковы ли оба значения.

Копирование объектов

1) Поверхностное копирование — для решения этой проблемы подходят `Object.assign` и оператор расширения. На практике второй вариант используется чаще из-за лучшей читаемости. Также необходимо учитывать, что для некоторых задач поверхностное копирование не только достаточно, но и необходимо.

2) Для глубокого копирования лучше всего подходит полный поиск объекта с использованием собственной функции или функции из библиотеки. Существует также «костыль» в виде `JSON.parse()` + `JSON.stringify()`. Однако он копирует только сериализуемые свойства, то есть теряет методы и неопределенные значения. Кроме того, у него есть проблемы с копированием дат.

Существует также относительно новая функция под названием `structuredClone()`, но она пока не получила большой популярности.

Сравнение объектов

Ситуация со сравнением практически та же — единственным надежным вариантом является специально адаптированная функция (рекурсивная в случае вложенных объектов).

```
1  const obj = { message: 'hello' };
2  const anotherObj = { message: 'hello' };
3
4  console.log(obj === obj); // true
5  console.log(obj === anotherObj); // false
6  console.log(Object.is(obj, anotherObj)); // false
```

Массивы

Массив, подобно объекту, представляет данные в виде определенного набора пар ключ: значение. Однако, если в обычных объектах основной упор делается на удобную работу со значениями по их именам (ключам), то в массивах данные располагаются в виде индексированного списка.

Индексы значительно упрощают доступ к значениям массива. Для этого достаточно указать нужный индекс в квадратных скобках сразу после имени соответствующего массива (например, fruits[2]).

Как правило, массивы наиболее удобны для использования в ситуациях, когда необходимо хранить какой-либо список данных одного типа. Хотя JS допускает возможность хранения значений разных типов в массиве, делать это крайне не рекомендуется!

В итоге можно сказать, что массив — это упорядоченная коллекция данных.

Итераторы

Итерация, по сути, означает прохождение (или итерацию). Итерируемые сущности — это сущности, которые можно обходить (или итерировать). В JavaScript концепция итерируемых сущностей появилась в спецификации ES6. С этого момента любая сущность, корректно реализующая функцию итерации с ключом [Symbol.iterator], является итерируемой.

В той же спецификации была введена новая реализация цикла for, специально разработанная для таких сущностей: for .. of. Кроме того, в ES6 появились и упомянутые ранее операторы расширения. По умолчанию итерируемыми сущностями являются все строки, массивы, а также объекты Map и Set.

Для реализации собственного итератора необходимо описать специальную функцию в объекте, используя ключ [Symbol.iterator]. Эта функция должна возвращать объект с методом next(), с помощью которого итерация выполняется напрямую. Метод next() также должен возвращать специальный объект, содержащий поле value — текущее значение итерации, и done — состояние завершения итерации (если done истинно, то итерация завершена). Чтобы создать массив из итерируемого объекта, достаточно вызвать встроенный метод Array.from(iterable).

Псевдо-массивы

В процессе работы нашего приложения мы сталкиваемся с объектами, которые очень похожи на массивы, но на самом деле не являются массивами. Такие объекты называются псевдо-массивами. По сути, это обычные объекты, в которых, как и в массиве, элементы расположены в упорядоченном виде (по индексам) и имеют свойство длины. В принципе, это всё, что объединяет массивы и псевдо-массивы, то есть у последних нет всех этих встроенных методов для работы с массивами.

Естественно, чаще всего, встречая псевдо-массив, мы хотели бы работать с ним как с полноценным массивом. Это можно сделать разными способами, но... Поскольку все псевдомассивы по умолчанию являются итерируемыми сущностями, проще всего использовать уже знакомый нам метод `Array.from(pseudoArray)`.

Конвертация объектов в примитивы

Первое и самое простое правило преобразования объектов в примитивы заключается в том, что любой объект, приведенный к логическому типу, всегда будет равен значению true.

В ситуациях, когда мы пытаемся сложить, вычесть или выполнить какую-либо другую (кроме сложения) математическую операцию над двумя объектами, происходит преобразование в число.

Ситуаций, когда происходит явное преобразование объекта в строку, не так много, но тем не менее они существуют. Так, например, при вызове функции браузера `alert(object)`, объект, переданный в качестве параметра, автоматически преобразуется в строку, или в случаях, когда мы пытаемся использовать объект в качестве ключа для значения другого объекта.

Cast Hints

Когда объект «понимает», что его необходимо привести к типу числа или строки, он, исходя из требуемого типа приведения, пытается вызвать внутренний метод, используя ключ [Symbol.toPrimitive], и передает этот тип приведения в качестве параметра. Возможные типы:

- ‘string’ — явное или неявное преобразование в строку;
- ‘number’ — явное или неявное преобразование в число;
- 'default’ — в случаях, когда оператор «не уверен», какой тип данных ожидать (например, при сложении, поскольку оно может использоваться как для чисел, так и для строк; при сравнении с приведением типов (==) и т. д.).

Как правило, приведение типов в случаях 'number' и 'default' дает одинаковый результат. Однако, если вы сами описываете логику приведения типов, то, естественно, вы можете сами решить, как объект будет вести себя в этих случаях.

Autoboxing

Как мы узнали ранее, в JavaScript не всё, что мы видим, является объектом. Зная это, может возникнуть довольно интересный вопрос: как мы получаем доступ к методам примитивных типов? В конце концов, примитивные типы — это всего лишь значения. Однако мы всё ещё можем использовать встроенный метод `.toString()` для чисел или метод `.slice()` для строк. Так откуда же они берутся?

Автоупаковка — это внутренний механизм JavaScript, суть которого заключается в том, чтобы обернуть примитивный тип в объект соответствующего типа (Number для чисел, String для строк) при попытке доступа к «его внутренним свойствам или методам».

За этим пугающим определением скрывается довольно простой процесс. Всякий раз, когда мы обращаемся к свойствам/методам нашего примитивного типа, мы фактически работаем со свойствами/методами временного объекта-обертки, который создаётся только в момент обращения и немедленно уничтожается.

Functions & context

NFE, IIFE

NFE — стандартное объявление Function Expression

IIFE (Immediately Invoked Function Expression) — функция, выполняющаяся сразу после объявления. Основная цель — создание приватной области (local scope)

```
1  const sayHi = function sayHiInnerRef(who) {
2      if (who) {
3          alert(`Hello, ${who}`);
4      } else {
5          sayHiInnerRef('Guest');
6      }
7  };
8
9  sayHiInnerRef('Roman'); // Error! No such function
```

```
(function () : void  {
    var data : {}  = data || {};
    console.log('This is IIFE', data)
})()
```

Анонимные функции

Анонимные функции — это все функции, которые не имеют имени. Всё очень просто. Такие функции обычно используются в качестве стрелочных функций обратного вызова. Это логично, поскольку в большинстве таких случаев нам просто нужно описать функцию, которая будет использоваться только здесь и сейчас и не будет использоваться дальше в коде.

```
1  const isDefined = function (value) {
2      return value !== null && value !== undefined;
3  }
4
5  const numbers = [1, 2, 3, 4, 5];
6  numbers.forEach(num => console.log(num));
7  numbers.filter(num => num > 3);
```

Callback-functions

Функция обратного вызова — это любая функция, переданная в качестве аргумента. Более «подробное» описание можно найти в интернете, но так или иначе, все они сводятся к одному и тому же.

Такие функции необходимы для их использования внутри других функций. До появления промисов они широко использовались для обработки HTTP-запросов. Сейчас, естественно, они утратили эту нишу, но, несмотря на это, до сих пор довольно часто используются. Хорошим примером использования может служить итерация по массивам.

```
1   const log100 = () => console.log(100);
2
3   setTimeout(log100, 1000);
4
5   function safelyExecuteFunction(callback) {
6       try {
7           callback();
8       } catch (error) {
9           console.log(error);
10      }
11  }
```

Конструкторы функций

Конструкторские функции — это специальные функции, предназначенные для создания (конструирования) новых объектов. Такие функции, как правило, не предназначены для прямого вызова, поскольку это не принесет большой пользы (хотя это все же можно сделать). Эти функции необходимо вызывать со специальным оператором `new`.

```
1   const uuid = require('uuid');
2
3   function User(firstName, lastName) {
4       this.id = uuid.v4();
5       this.firstName = firstName;
6       this.lastName = lastName;
7
8       this.getFullName = function() {
9           return `${this.firstName} ${this.lastName}`;
10      }
11
12      this.startWorking = function() {
13          console.log(`User ${this.getFullName()} started to work...`);
14      }
15
16      this.stopWorking = function() {
17          console.log(`User ${this.getFullName()} stopped to work...`);
18      }
19  }
```

Higher Order Functions

Функции высшего порядка — это функции, которые либо принимают другие функции в качестве аргументов (работа с функциями обратного вызова), либо возвращают функцию в качестве результата. В качестве примера можно привести итерацию по методам массивов или метод `bind` — встроенный метод, возвращающий функцию в качестве результата.

Функции высшего порядка не могут существовать в языках, не поддерживающих механизм первоклассных объектов (функций). Первоклассная функция — это функция, которая может выступать в качестве аргумента функции, быть возвращаемым значением функции и выступать в качестве значения для переменной или поля объекта.

Локальные переменные

Говоря о функциях высшего порядка, нельзя игнорировать тему замыканий в JavaScript. Но прежде чем перейти к самим замыканиям, необходимо понять понятия локальных и глобальных переменных.

Локальные переменные — это переменные, созданные в пределах некоторой ограниченной области видимости. Для переменных, созданных с помощью let или const, эта область представляет собой блок. Для переменных, созданных с помощью var, это функция.

Локальные переменные существуют только в пределах своей области видимости. За её пределами доступ к ним невозможен (будет сгенерирована ошибка).

```
1   console.log(innerVariable); // Error! No such variable
2
3   function someFuncWithInnerVariable() {
4       var innerVariable = 100;
5   }
6
7   console.log(innerVariable); // Error! No such variable
8
9   const numbers = [1, 2, 3, 4, 5];
10
11  for (let index = 0; index < numbers.length; index++) {
12      console.log(numbers[index]); // 1 2 3 4 5
13  }
14
15  console.log(index); // Error! No such variable
```

Глобальные переменные

Глобальные переменные — это любые переменные, созданные за пределами какой-либо области видимости. Кроме того, к глобальным переменным относятся поля глобального объекта (window для браузеров, global для Node.js и т. д.).

```
1   const MAX_TEMPERATURE = 30;
2   const MIN_TEMPERATURE = -25;
3
4   function calcDiffWithMaxTemperature(temperature) {
5       return Math.abs(MAX_TEMPERATURE - temperature);
6   }
7
8   function calcDiffWithMinTemperature(temperature) {
9       return Math.abs(MIN_TEMPERATURE - temperature);
10  }
```

Lexical environment

Лексическое окружение — это особый объект, хранящий информацию обо всех локальных переменных и ссылку на внешнее окружение (внешнее лексическое окружение). Этот объект скрыт и недоступен напрямую. Тем не менее, в современных реалиях можно четко выделить два его ключевых компонента: запись окружения — объект, хранящий все локальные переменные (и некоторую другую информацию, например, значение переменной `this`), и внешний объект — ссылка на внешнее лексическое окружение.

По сути, все локальные переменные являются скрытыми объектными переменными внутри лексического окружения. То есть, при попытке получить доступ к переменной, первым шагом является обращение к этому объекту, чтобы получить значение с этим именем оттуда, а затем (если это не удается) обращение к внешней скрытой ссылке на объект переменных внешнего окружения. Цикл повторяется до тех пор, пока: а) переменная не будет найдена, б) цепочка внешних окружений не завершится.

Замыкания (closure)

1) Замыкание — это функция, которая запоминает свои внешние переменные и может к ним обращаться.

2) Замыкание — это комбинация функции и лексической среды, в которой эта функция была определена.

В JavaScript почти все функции являются замыканиями.

```javascript
let firstName = 'Roman';

function logFirstName() {
    console.log(firstName);
}

function makeFirstNameLogger() {
    const firstName = 'Vladimir';
    return function() {
        console.log(firstName);
    }
}

const customFirstNameLogger = makeFirstNameLogger();
customFirstNameLogger(); // 'Vladimir'

logFirstName(); // 'Roman'
firstName = 'Darya';
logFirstName(); // 'Darya'
```

Контекст выполнения

Контекст выполнения в JavaScript используется для отслеживания хода выполнения кода. Именно с его помощью определяется доступная среда на текущем этапе выполнения программы. Контекст выполнения также содержит дополнительные параметры, которые генерируются независимо движком JavaScript при обработке вашего кода.

Контекст выполнения, как и, например, лексическая среда, является абстрактным механизмом спецификации. Это означает, что к нему нельзя получить прямой доступ.

По сути, контекст выполнения просто предоставляет своего рода обертку или среду для исполняемого кода, содержащую некоторые вспомогательные элементы для отслеживания состояния программы, некоторые из которых даже доступны непосредственно из кода (например, значение специального объекта this).

Глобальный контекст выполнения

Глобальный контекст выполнения — это контекст выполнения по умолчанию. В этом контексте весь начальный код JavaScript выполняется при загрузке файла в браузер. Любой глобальный код, то есть код, не находящийся внутри функции, выполняется в глобальном контексте выполнения.

JavaScript допускает только одну глобальную среду. Это означает, что JavaScript не может иметь более одного контекста выполнения. Это происходит из-за однопоточной архитектуры языка.

```javascript
// The code below is being executed in global execution context
var oneHundred = 100;

function log100() {
    console.log(100);
}

// Variables created with key word 'var' and
// 'function declarations' are being saved in
// global object as its properties values
console.log(window.oneHundred); // 100
console.log(window.log100); // f log100()
```

Локальный (функциональный) контекст выполнения

Функциональный контекст выполнения — это контекст выполнения, создаваемый языком в момент вызова функции.

Каждая функция имеет свой собственный контекст выполнения, поэтому их может быть несколько. Функциональный контекст выполнения имеет доступ ко всему коду в глобальном контексте выполнения.

Если JavaScript видит вызов функции во время выполнения кода в глобальном контексте выполнения, он создает для неё новый контекст выполнения.

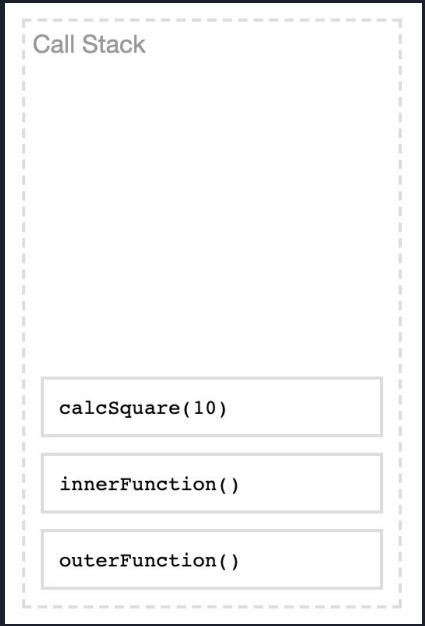
```
1  function outerFunction() {
2      const innerCalculation = 10 + 35;
3      // In line 5 execution context for method 'log' from 'console'
4      // object is being created
5      console.log(innerCalculation);
6      // In line 8 execution context for function 'innerFunction'
7      // is being created
8      return innerFunction();
9
10     function innerFunction() {
11         // In line 13 execution context for function 'calcSquare'
12         // is being created
13         return calcSquare(10);
14     }
15 }
16
17 // In line 19 execution context for function 'outerFunction'
18 // is being created
19 const result = outerFunction();
20 // In line 22 execution context for method 'log' from 'console'
21 // object is being created
22 console.log(result);
23
24 function calcSquare(x) {
25     return x * x;
26 }
```

Call Stack & Context

Call Stack

Для хранения и отслеживания контекстов выполнения они формируются в стек контекстов выполнения — список контекстов, организованных по принципу LIFO («последний вошел — первый вышел»).

Текущий (выполняемый) контекст всегда является верхним элементом стека. После выполнения необходимого кода связанный с ним контекст выполнения удаляется. Управление возвращается к контексту, который был элементом ниже, и теперь он будет верхним элементом, то есть текущим контекстом выполнения.



Call Context

Контекст вызова, по сути, представляет собой значение специального объекта `this`. Это значение обычно зависит от способа вызова функции.

Любая функция, вызываемая «сама по себе», будет иметь в качестве значения `this` ссылку на глобальный объект или `undefined`.

Если функция вызывается как метод объекта, значение `this` будет ссылкой на этот объект.

Если функция вызывается через `new`, значение `this` будет ссылкой на новый объект.

```
1   function logThis() {
2       console.log(this);
3   }
4
5   logThis(); // globalThis
6
7   const obj = {
8       value: 100,
9       logThis,
10  };
11
12  obj.logThis(); // obj
13
14  function User() {
15      console.log(this);
16  }
17
18  new User(); // {} (empty object)
```

Loss of context (Потеря контекста)

Довольно часто начинающие (а иногда и не только начинающие) JavaScript-разработчики сталкиваются с проблемой потери контекста. Это нормально и довольно часто встречается на практике.

Если запустить код из примера в среде Node.js, то через 1 секунду мы получим в качестве результата следующую строку: undefined только что закрыл приложение!

```
1   const user = {
2       firstName: 'Roman',
3       closeApp() {
4           console.log(
5               `${this.firstName} has just closed the app!`
6           );
7       },
8   };
9
10  setTimeout(user.closeApp, 1000);
```

call, apply, bind

```
function logSurnameAndThisName(surname) {
    console.log(surname);
    console.log(this.name);
}

const user = {
    name: 'Roman',
};

logSurnameAndThisName.call(user, 'Alexanov'); // Alexanov Roman
```

```
function logSurnameAndThisName(surname) {
    console.log(surname);
    console.log(this.name);
}

const user = {
    name: 'Roman',
};

logSurnameAndThisName.apply(user, [ 'Alexanov' ]); // Alexanov Roman
```

```
function logSurnameAndThisName(surname) {
    console.log(surname);
    console.log(this.name);
}

const user = { name: 'Roman' };

const logAlexanovAndUserName = logSurnameAndThisName.bind(user, 'Alexanov');
logAlexanovAndUserName(); // Alexanov Roman

user.name = 'Vladimir';
logAlexanovAndUserName(); // Alexanov Vladimir
```